UML:

Rapport Projet POO – Version 3 (POO-v3)

📁 1. Import et export CSV

- Création d'une méthode `importListAdo(String path)` dans la classe **`File`** pour lire un fichier CSV et charger les adolescents (invités ou hôtes).
- Création de la méthode `export(String path, Set<Adolescent> invites, Set<Adolescent> hotes)` pour écrire les paires appariées dans un fichier CSV (prénom, nom, score).

🔁 2. Gestion de l'historique

- Création de la classe **`Historique.java`** pour gérer la sérialisation et désérialisation d'un `HashMap<Adolescent, Adolescent>` représentant les appariements passés.
- Ajout d'une surcharge de la méthode **`Appariement.apparier(...)`** avec un paramètre `Map<Adolescent, Adolescent> historique` permettant :
    - De refuser un ancien correspondant si le critère **`HISTORY`** est *`other`*.
    - De forcer un ré appairage si le critère est *`same`*.
- Gestion des cas où aucun historique n'est fourni (cas d'un premier séjour).

✋ Modifications apportées

✅ Refactorings

- Suppression de l'attribut *`est_hote`* dans la classe **`Adolescent`**, devenu inutile grâce à la distinction explicite via deux fichiers CSV (un pour les invités, un pour les hôtes).

.

UML:

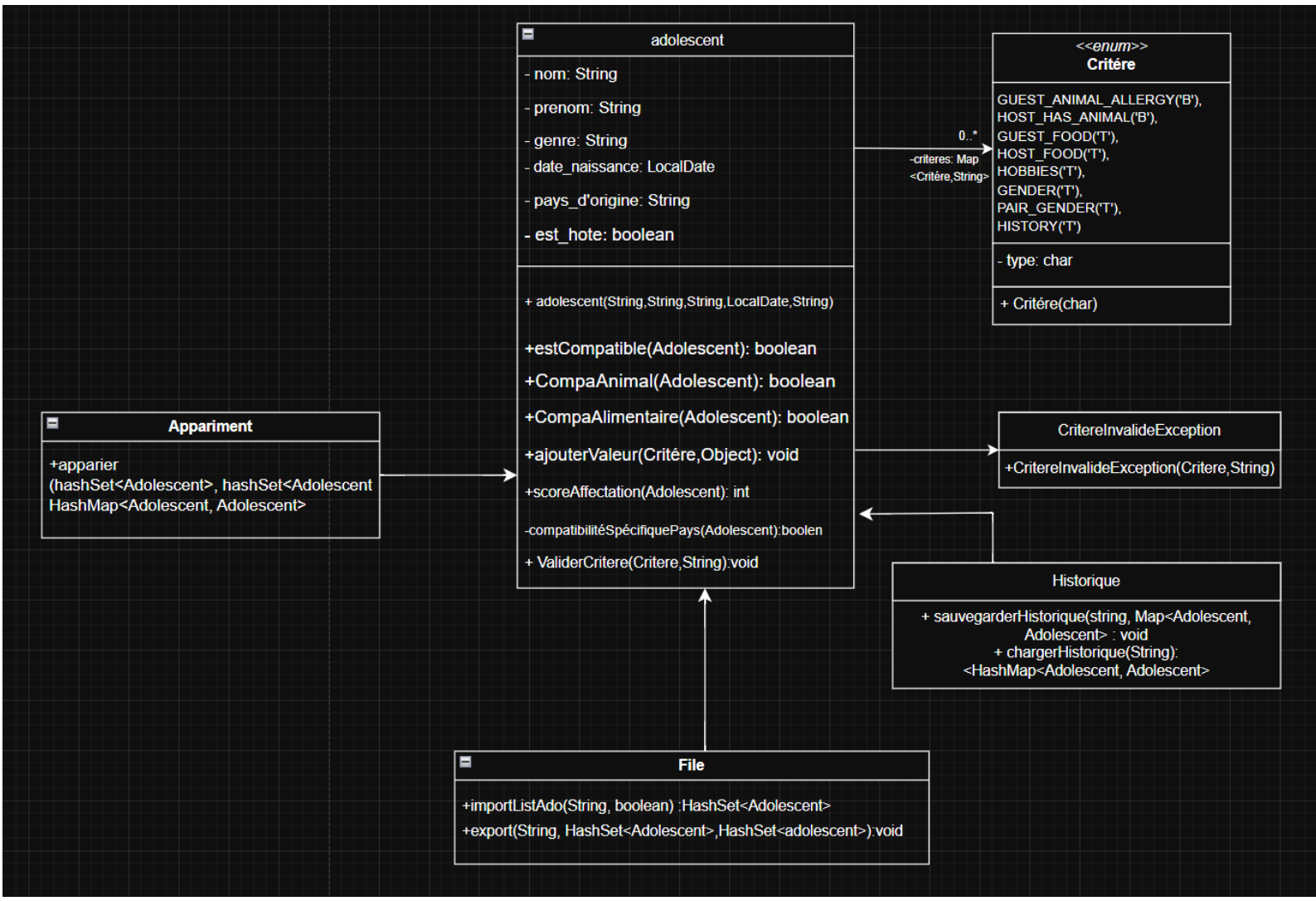
💡 Améliorations de robustesse

- Correction des erreurs liées aux valeurs vides dans le fichier CSV.
- Affichage des appariements trouvés avec gestion explicite des cas où un adolescent n'a pas pu être apparié.
- Vérification de la validité des critères via la classe `CritereInvalideException`.

✅ Fonctionnalités opérationnelles

- Import CSV des invités et des hôtes (dans deux fichiers séparés).
- Calcul d'un score d'affinité basé sur :
    - Contraintes rédhibitoires (allergies, régime alimentaire, `HISTORY`)
    - Préférences (genre, hobbies, âge)
- Appariement optimal par score le plus bas
- Export CSV du résultat
- Historique sérialisé utilisable pour les prochaines affectations

UML:


adolescent
- nom: String
- prenom: String
- genre: String
- date_naissance: LocalDate
- pays_d'origine: String
- est_hote: boolean
+ adolescent(String,String,String,LocalDate,String)
+estCompatible(Adolescent): boolean
+CompaAnimal(Adolescent): boolean
+CompaAlimentaire(Adolescent): boolean
+ajouterValeur(Critére,Object): void
+scoreAffectation(Adolescent): int
-compatibilitéSpécifiquePays(Adolescent):boolen
+ ValiderCritere(Critere,String):void
0..*
-criteres: Map
<Critére,String>
<<enum>>
Critére
GUEST_ANIMAL_ALLERGY('B'),
HOST_HAS_ANIMAL('B'),
GUEST_FOOD('T'),
HOST_FOOD('T'),
HOBBIES('T'),
GENDER('T'),
PAIR_GENDER('T'),
HISTORY('T')
- type: char
+ Critére(char)
Appariment
+apparier
(hashSet<Adolescent>, hashSet<Adolescent
HashMap<Adolescent, Adolescent>
CritereInvalideException
+CritereInvalideException(Critere,String)
Historique
+ sauvegarderHistorique(string, Map<Adolescent,
Adolescent> : void
+ chargerHistorique(String):
<HashMap<Adolescent, Adolescent>
File
+importListAdo(String, boolean) :HashSet<Adolescent>
+export(String, HashSet<Adolescent>,HashSet<adolescent>):void